

ARTICLE

The Recognition-Exposure Gap: Why Upgrading Intent Detection Can Reduce Tool Availability in LLM Agents

Anonymous Author(s)

Anonymous affiliation withheld for double-blind review

Correspondence

Correspondence concerning this manuscript should be directed to the handling editor as part of the double-blind review process.

Abstract

Tool-rich LLM agents gate tool visibility by intent-detection confidence, exposing a narrow set when confident, a broad set otherwise—assumed monotone in detector quality. We report a counterintuitive failure. In a production desktop agent (WeClaw, 125 tools), an LLM detector raised recommendation accuracy to 98.2% (491/500 queries, ground truth fixed by construction) yet *lowered* exposure coverage to 42.2%: in 56.2% of cases the correct tool was named but never entered the function-calling schema. We name this the **Recognition-Exposure Gap**: two decoupled stages—confidence-to-tier mapping and exposure-set construction—that a confidence-shifting upgrade leaves desynchronized, dropping recognized tools. A deterministic replay and a live run quantify the cost: 788 failed round-trips, 499 escalations, $3.7\times$ exposure inflation, and $2.6\times$ per-query latency. A zero-training fix that merges recognized tools into the exposure set closes the gap (reachability 42.2%→98.4%) and lowers exposure. The condition is defined detector- and framework-independently; we support this with blind second annotation ($\kappa = 0.987$), three-detector replication, and open-source reproduction.

KEYWORDS

tool exposure; LLM agents; function calling; intent detection; agent harness

1 | INTRODUCTION

As LLM agents gain access to hundreds of tools, exposing all of them at once becomes prohibitively expensive: full-schema prompts inflate token cost, latency, and selection error. A common remedy is **confidence-tiered exposure**—the harness runs an intent detector over the user’s request, maps the detector’s confidence to a visibility tier (a narrow *recommended* set at high confidence, a broader *extended* or *full* set otherwise), and exposes only that tier’s tools to the model’s function-calling interface. This design is explicitly “confidence-gated” in recent systems [1,2] and underlies a broad “route before reasoning” family [3,4].

The design carries an implicit assumption: **exposure quality is monotone in detector quality**. If the intent detector improves—say, from keyword rules to an LLM—the correct tool should become *more* reachable, not less. However, this paper shows the assumption can fail silently. In WeClaw [5], a production daily-use desktop agent with 125 registered tools, we upgraded the intent detector from a rule-based matcher to an LLM-based classifier. The upgrade behaved essentially as

intended at the recognition stage: on a 500-query set of colloquial, ambiguous user queries (25 tools, ground truth fixed *by construction*), the LLM recommended the correct tool in **98.2%** of cases (491/500; Wilson 95% CI [96.6, 99.1]). Yet the fraction of those queries for which the recommended tool actually appeared in the function-calling exposure set was only **42.2% (211/500)** [37.9, 46.6]. In the remaining **56.2%** (281/500) [51.8, 60.5], the agent had correctly recognized what the user needed and then failed to make the corresponding tool callable. We call this divergence the **Recognition-Exposure Gap**.

The cause is structural, not a coding mistake. Confidence-tiered exposure is implemented as two stages: **Stage A** maps detector confidence to a tier; **Stage B** constructs the concrete tool set for that tier. When the detector is upgraded, its confidence distribution shifts—in our case the LLM returns a fixed high confidence (0.9) on success, collapsing every ambiguous query into the *narrowest* tier. Stage B, however, still builds that tier’s tool set from the *rule-driven* intent-to-tool mapping, which is empty for colloquial queries; the LLM’s recommended tools are recorded as annotations and prompt hints but are **not merged into the exposure set**. The two stages are

decoupled: a recognition improvement changes Stage A’s output without propagating to Stage B’s construction, so correctly recognized tools are silently dropped (Fig. 1). Consequently, recognition accuracy—the metric everyone watches—cannot see this.

Moreover, the gap is not a benign offline curiosity. We quantify its runtime consequence two ways. A *deterministic replay* faithfully drives the production failure-escalation ladder (a tool call that cannot be resolved triggers `report_failure`, which after a threshold escalates the tier and re-exposes tools): across the 289 detour queries (of which 281 are gap queries) it incurs **788 failed round-trips** and **499 tier escalations**, inflating the mean exposed tool set from 12.1 at the first round to **44.7** at the final tier. A separate *real end-to-end run*—every round genuinely calling the LLM and measuring wall-clock—confirms the same asymmetry at true cost: mean per-query latency **11.2 s before** $\rightarrow$ **4.3 s after** (a $2.6\times$ reduction), with mean round-trips $2.58 \rightarrow 1.03$. The agent eventually does the right thing—but late, and at a cost. We then show a **zero-training co-design fix**: merging recognized tools into the exposure set at construction time closes the gap (first-attempt reachability $42.2\% \rightarrow 98.4\%$ live, 100% under deterministic replay; failed round-trips and escalations $\rightarrow 0$) and, because it avoids escalation to the broad tier, keeps the exposed set near its first-round size (mean $12.1 \rightarrow 13.2$ offline) rather than inflating it to 44.7.

Contributions. (1) We *name* the Recognition-Exposure Gap, a failure mode localized at the *interface between two harness modules* rather than in the model’s representations, and show it occurs even when recognition is $\sim 98\%$ correct on a 500-query set. (2) We contribute a *reusable coverage-vs-recognition audit method*: a black-box, three-probe protocol (offline counterfactual, deterministic escalation replay, and live end-to-end run) that measures the divergence between what a harness recognizes and what it actually exposes, applicable to any confidence-tiered exposure system. Using it, we quantify the gap’s runtime cost—a deterministic replay of the production escalation engine (reproducing round2’s 42.2% coverage / 56.2% gap) and an independent live run that calls the LLM every round and measures wall-clock latency. (3) We give a zero-training co-design fix—an *existence proof that the gap is closable at construction time* (first-attempt reachability $42.2\% \rightarrow 98.4\%$ live, 100% under deterministic replay)—and abstract a failure condition that is *defined* detector- and framework-independently, substantiating it with a second-annotator agreement study (Cohen’s $\kappa = 0.987$), replication across three heterogeneous LLM intent detectors, and a reference reproduction of the same two-stage decoupling inside three common open-source exposure strategies. We position the result as, to our knowledge, the first quantified tool-exposure instance of the model-harness “coupling bottleneck” [6].

2 | BACKGROUND: CONFIDENCE-TIERED TOOL EXPOSURE

System. WeClaw [5] is an open-source desktop LLM agent for daily personal use, with 125 registered tools spanning shell, file, screen, and search (the four *core* tools always exposed), plus ~ 23 *extended* tools (browser, clipboard, calculator, date-time, notification, app control, stock query, enterprise lookup, and others) and a long tail of domain tools.

Tiered exposure. A ToolExposureEngine maps an intent result to a visibility tier by confidence: confidence $\geq 0.8 \rightarrow$ RECOMMENDED (narrowest), $\geq 0.5 \rightarrow$ EXTENDED, else FULL. For each tier it constructs a concrete tool-name set: RECOMMENDED = core tools $\cup$ intent-to-tool mapping for the recognized intents $\cup$ a capability-info tool; EXTENDED adds the ~ 23 extended tools; FULL either exposes all registered tools or, under a retrieval-based fallback, a lexical Top-K union with a conservative floor. The constructed set is resolved for dependencies and is rendered into function-calling schemas—the hard boundary of what the model can call.

Failure escalation. The engine maintains a per-task consecutive-failure counter. When a tool call cannot be resolved, `report_failure` increments it; once it reaches a threshold (2 in production), `_upgrade_tier` widens the tier (RECOMMENDED $\rightarrow$ EXTENDED $\rightarrow$ FULL $\rightarrow$ unbounded) and the next attempt re-exposes tools at the wider tier. This ladder is the runtime safety net that recovers from an under-exposed first attempt—at the cost of extra round-trips.

The two stages. We separate, for analysis, **Stage A** (confidence $\rightarrow$ tier) from **Stage B** (tier $\rightarrow$ tool set construction). The Recognition-Exposure Gap arises precisely because these stages are decoupled: the detector’s recommended tools influence Stage A’s confidence but, absent an explicit merge, do not influence Stage B’s construction.

3 | THE RECOGNITION-EXPOSURE GAP

Let a query carry a ground-truth target tool ref (the tool a correct assistant should call). Define two booleans produced by *different* pipeline stages:

- `rec_hit` = the intent detector **recommended** ref (Stage A / recognition);
- `exposed` = `ref` $\in$ the **function-calling exposure set** (Stage B / construction).

The **Recognition-Exposure Gap** is the event `rec_hit  $\wedge$   $\neg$  exposed`: the agent recognized the needed tool but did not

make it callable. $P(\text{rec_hit})$ is reported by recognition accuracy and $P(\text{exposed})$ by exposure coverage; the gap is their divergence. As a result, the gap is invisible to standard evaluations, which report only recognition accuracy.

General failure condition. The gap arises whenever a harness combines (a) **confidence-tiered exposure**, where a detector-confidence shift can move queries into a narrower tier, with (b) **decoupled recommended-tool injection**, where the detector’s recommended tools are not merged into the tier’s constructed set. Under (a)+(b), *any* detector upgrade that raises confidence—paradoxically, an improvement—can push ambiguous queries into the narrowest tier while the recommended tools remain outside it. Moreover, the condition is *defined* detector-independently (rules, LLM, or hybrid) and framework-independently (any harness with the two-stage structure). We verify it on a production LLM detector, replicate the recognition stage across three heterogeneous LLM detectors, isolate the mechanism via deterministic replay, and reproduce the same decoupling inside three common open-source exposure strategies (Section 5).

Terminology. We use *tool exposure* in its **capability** sense—the set of tools a harness makes visible/callable to the model—strictly distinct from *data over-exposure* in the privacy sense [7], which concerns an agent transmitting sensitive data beyond functional necessity. The two are in fact opposite failure directions of the same interface: over-exposure risks privacy, **under-exposure** (this work) risks capability. We return to this framing in Section 6.

4 | METHOD: MEASURING AND CLOSING THE GAP

4.1 Offline counterfactual (round2). We reproduce the production LLM-intent path: for each query, an LLM classifier returns an intent result with recommended tools, and whether *ref* is in the exposure set the engine actually constructs is checked. This yields *rec_hit*, *exposed*, and thus the gap without executing tool calls—the fast, first-order measurement. We run it on a 500-query set (25 tools, up to 20 queries per tool, with the ground-truth *ref* fixed *by construction* from each query’s seeded intent); every reported rate carries a Wilson 95% confidence interval.

4.2 Deterministic escalation-ladder replay (round3). In order to measure the gap’s *runtime* consequence, we faithfully replay the escalation ladder. For each query we drive the real production code path (`_determine_tier` → `_get_tool_names_for_tier` → dependency resolution) round by round: if *ref* ∉ *exposed* set, a failed round-trip is recorded (the semantics of an unresolvable first function call) and `report_failure` is called, which escalates the tier once

the threshold is reached; tools are exposed again and the round is repeated until *ref* becomes reachable (recovery) or the ladder tops out. In order to eliminate API nondeterminism and cost, round3 is a **deterministic replay**: it injects round2’s measured recommendation outcome (`llm_recommended_tools` = [*ref*]) rather than re-calling the LLM, while exposure construction and escalation are performed through the unmodified production engine. We run two arms on the same queries: **before** (merge off, the pre-fix behavior) and **after** (merge on, the fix).

4.3 Zero-training co-design fix. The fix merges recognized tools into the exposure set at construction time: `_augment_with_llm_recommended` inserts `llm_recommended_tools` into both the FULL-fallback and non-FULL return points of Stage B, under four guards (config gate; known-tool filtering against the registry; forbidden-tool precedence; a `max_merge` cap of 8). In other words, the fix simply connects Stage A’s output to Stage B’s construction: no model is trained and no threshold is retuned.

4.4 Real end-to-end run (live). Because the deterministic replay (4.2) fixes recognition and uses a per-round-trip latency proxy, we separately run a *live* variant that removes both idealizations: on the same 500 queries, every round makes a genuine function-calling request to the production LLM (`deepseek-v4-flash`) against the currently exposed schemas, and we measure true wall-clock latency. The model’s own tool choice (not an injected *ref*) drives whether a round resolves; a miss triggers the real `report_failure/_upgrade_tier` ladder exactly as in production. To make the cost measure independent of client-side concurrency, we report the *accumulated per-round-trip model latency* (sum of measured round-trip times per query), not wall-clock queueing time. This confirms the replay’s before/after asymmetry under real API nondeterminism and real latency.

4.5 Validation protocols. Three protocols stress-test the headline claim. (i) *Blind second annotation*: because *ref* is fixed by construction, we cross-check it against an independent, different-provider LLM annotator (`qwen-max`) that, from the query and tool list alone, picks the tool a correct assistant should call; we report raw agreement and Cohen’s κ . (ii) *Multi-detector replication*: we re-run the recognition stage with two additional different-provider detectors (`qwen-max`, `glm-5`) on a stratified subsample; since the merge-off exposure set is fixed by the rule intent (detector-independent), any cross-detector difference is attributable to recognition alone. (iii) *Reference reproduction on open-source strategies*: we implement three common exposure strategies—an intent-router with a static allow-list, an all-tools full-bind, and a Top-K tool retriever—under the same recognition signal, with and without the merge fix, to test whether the gap is a property of the two-stage architecture rather than of WeClaw specifically.

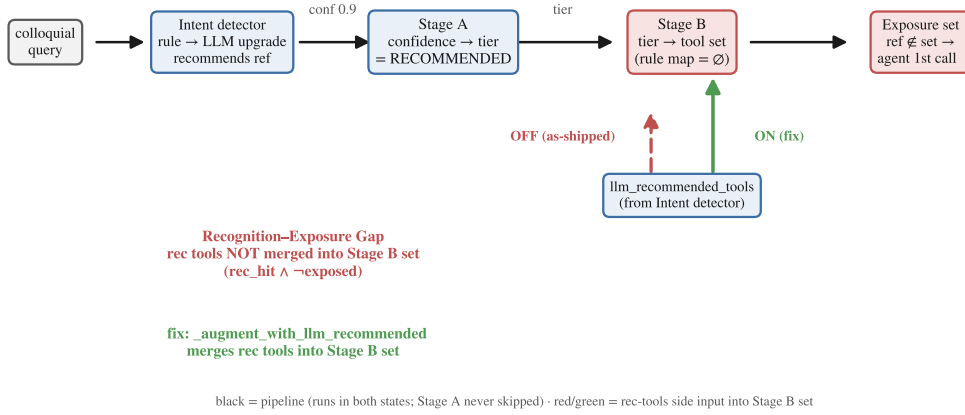


FIGURE 1 Mechanism of the Recognition-Exposure Gap. The intent detector’s recommended tools drive Stage A’s confidence (solid arrow) but are not merged into Stage B’s tool-set construction (dashed, ×), so a correctly recognized tool never enters the function-calling exposure set; the zero-training fix (green) reconnects the two decoupled stages.

5 | EXPERIMENTS

5.1 Setup. The evaluation set is 500 colloquial, ambiguous Chinese queries spanning 25 tools (up to 20 per tool; e.g., “现在几点了” / “what time is it now” → `datetime_tool`, “帮我算个数” / “help me do a calculation” → `calculator`, “弹个通知提醒我” / “pop me a notification” → `notify`); each query’s ground-truth `ref` is fixed *by construction* from its seeded intent, so the label is independent of any detector. The intent LLM is `deepseek-v4-flash`, which on success returns confidence 0.9 (collapsing queries into `RECOMMENDED`). Escalation threshold = 2 (production value); for the deterministic replay the per-failed-round-trip cost proxy is 1,950 ms (a lower bound, since a real round-trip also includes tool execution and second-pass generation), while the live run (Section 5.6) measures true wall-clock (~4.16 s mean per round-trip). Registered tools = 125. All rates carry Wilson 95% confidence intervals.

5.2 Offline counterfactual (round2). Table 1 and Fig. 2 report the recognition/exposure divergence. The LLM recommends the correct tool in 98.2% of cases. However, fewer than half of those recommended tools are actually exposed; merging recognized tools lifts counterfactual coverage to 98.4%.

5.3 Deterministic replay results (round3). Table 2 and Fig. 3 contrast the two arms under deterministic replay. Before the fix, only 42.2% of queries can call the recommended tool on the first attempt; the other 57.8% must detour through the escalation ladder, and all of them do eventually recover. In aggregate the detour costs **788 failed round-trips** and **499 tier escalations** across the 289 detour queries (281 of them gap queries), inflating the mean exposed set from 12.1 (first

TABLE 1 Offline counterfactual measurement (round2), LLM intent detection, 500 queries whose ground-truth `ref` is fixed by construction; 95% Wilson CIs in brackets. The tier collapse and exposure drop are deterministic code-path consequences; the 98.2% recognition hit is a real LLM measurement, and blind second annotation agrees at $\kappa = 0.987$ (Section 5.7). Exposure sizes are the offline constructed set (cf. Table 2’s dependency-resolved first-attempt sizes).

Metric	Value
LLM recommendation hit (<code>rec_hit</code>)	98.2% (491/500) [96.6, 99.1]
Real exposure coverage (<code>exposed</code>)	42.2% (211/500) [37.9, 46.6]
Recognition-Exposure Gap	56.2% (281/500) [51.8, 60.5]
Coverage <i>with merge fix</i>	98.4% (492/500) [96.9, 99.2]
Mean exposure size (before → after)	12.1 → 13.2 tools

round) to 44.7 at the final tier and accumulating a 1,536,600 ms latency lower bound (~5.3 s per detour query). After the fix, first-attempt reachability is 100%, with zero failed round-trips, zero escalations, and zero added latency; mean exposure stays at 12.7 instead of escalating to 44.7, because no query is forced up to the broad tier (the fix adds only the recognized tool at the first attempt but avoids the blow-up).

Table 3 and Fig. 4 show representative trigger chains from the before arm. The pattern is uniform: two failed attempts inside the `RECOMMENDED` tier (the recommended tool is absent), then one escalation to `EXTENDED`, where the tool finally appears. Control queries whose `ref` is a core/always-resident tool

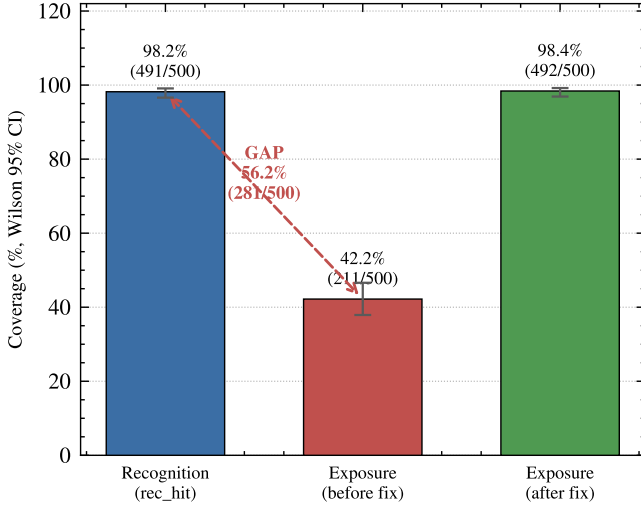


FIGURE 2 Recognition vs. exposure coverage (round2, 500 queries with a ground-truth tool fixed by construction; error bars are Wilson 95% CIs). The LLM recommends the correct tool in 98.2% of cases, yet only 42.2% of those tools are actually exposed (gap 56.2%); merging recognized tools into the exposure set restores 98.4% coverage.

TABLE 2 Deterministic escalation-ladder replay (round3), before/after arms on the same 500 queries (escalation threshold 2). First-attempt sizes are dependency-resolved, hence 12.1 matching Table 1’s offline first-round mean. The after-arm merges only the ground-truth ref per query, so its exposure (12.7) is a lower bound; with the full recommended list the offline mean is 13.2 (Table 1).

Metric	before (merge off)	after (merge on)
First-attempt reachability	42.2% [37.9, 46.6]	100% [99.2, 100]
Need-detour rate	57.8% [53.4, 62.1]	0%
Recovery after detour	100% (289/289)	n/a
Failed round-trips (total)	788	0
Tier escalations (total)	499	0
Mean first-round exposure	12.1	12.7
Mean final-tier exposure	44.7	12.7
Extra latency LB (cumulative)	≈5.3 s/detour query	0

(file, search, capability-info) reach on the first attempt in both arms, confirming the chain does not over-report.

5.4 Floor ablation (round0/round1). A separate ablation (Fig. 5) isolates the extended-tool floor of the FULL tier. Removing the floor causes **50%** of ambiguous queries to miss a high-frequency tool on the first attempt in the FULL tier (e.g., datetime, calculator, browser); back-filling colloquial keywords into tool descriptions drops the escalation trigger

rate from **50%** → **0%** and retrieval zero-hits from 4/20 → 1/20. Thus, this confirms the gap’s mechanism is about *which tools enter the constructed set*, and that the failure condition is not specific to the LLM detector—any confidence distribution that lands ambiguous queries in a narrow tier without the recommended tools reproduces it.

5.5 Cross-validation. The three execution-side methods agree: round3’s deterministic first-attempt reachability (42.2% before / 100% after) matches round2’s real exposure coverage (42.2%) and gap (56.2%), and the live end-to-end run (Section 5.6) independently reproduces the same 42.2% first-attempt split under genuine per-round LLM calls. The replay and live probes re-derive exposure and escalation through the unmodified production engine while taking round2’s measured recognition outcome as a given input (the replay does not re-call the LLM; the live run does), so they are not statistically independent at the recognition stage. Nevertheless, they independently confirm the exposure-construction and escalation behavior, strengthening confidence that the gap is a real property of the pipeline rather than an artifact of either probe.

5.6 Real end-to-end run (live). Table 4 reports the live arm, in which every round issues a genuine deepseek-v4-flash function-calling request against the currently exposed schemas and wall-clock latency is measured. The first-attempt split (42.2% before / 98.4% after) reproduces the offline coverage, and both arms eventually reach 100%. The fix’s effect on cost is direct: mean round-trips per query fall from 2.58 to 1.03 and mean accumulated model latency from 11.2 s to 4.3 s (a 2.6× reduction; medians 7.4 s → 1.4 s), while total LLM calls fall from 1,288 to 517. Notably, when the correct tool is exposed the model selects it at essentially the same rate in both arms (71.0% before / 72.8% after choosing ref as its first call), confirming that the before-arm failures are *exposure* failures, not *selection* failures—the model calls whatever it can see, and under the gap the right tool is simply not visible.

5.7 Label independence (blind second annotation). Because ref is fixed by construction, the recognition metric could in principle be circular. We therefore cross-check every query’s ref against an independent, different-provider annotator (qwen-max) that, seeing only the query and the tool list, picks the tool a correct assistant should call. The two labels agree on **98.8%** of the 500 queries (Cohen’s $\kappa = 0.987$); the six disagreements are all genuinely polysemous queries (a request satisfiable by either of two tools), not systematic mislabels. This shows the ground truth is not an artifact of the primary detector, so the reported 98.2% recognition hit and 56.2% gap are not self-fulfilling.

5.8 Robustness. Two protocols test whether the gap is an artifact of one detector or of WeClaw.

Multi-detector replication (Table 5). We re-run recognition on a 150-query stratified subsample with two additional

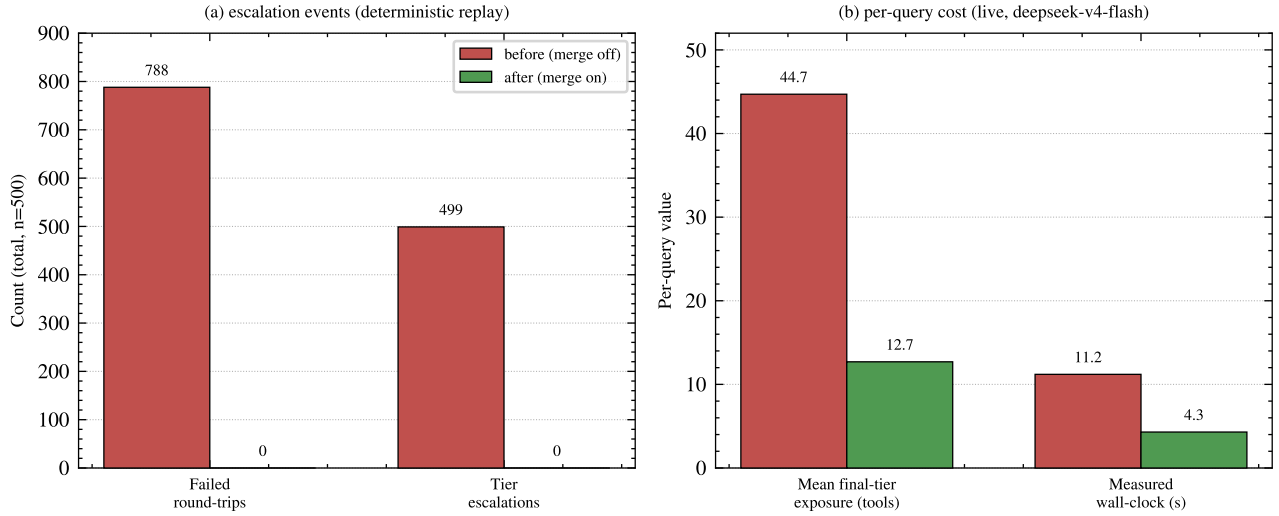


FIGURE 3 Runtime cost, before vs. after the fix. (a) Deterministic replay over 500 queries: the gap incurs 788 failed round-trips and 499 tier escalations, both driven to 0 by the fix. (b) Per-query cost: mean final-tier exposure inflates from 12.1 to 44.7 tools under the ladder (12.7 after the fix), and a real end-to-end run measures mean accumulated latency 11.2 s before vs. 4.3 s after ($2.6\times$).

TABLE 3 Representative before-arm trigger chains (rt = round-trip; tier, lexposedl, whether `ref` is reachable).

Query (zh / en gloss)	ref	Trajectory
现在几点了 / “what time is it now”	datetime_tool	rt1[REC, 5, $\times$] $\rightarrow$ rt2[REC, 5, $\times$] $\rightarrow$ rt3[EXT, 31, $\checkmark$]
帮我算个数 / “help me do a calculation”	calculator	rt1[REC, 5, $\times$] $\rightarrow$ rt2[REC, 5, $\times$] $\rightarrow$ rt3[EXT, 31, $\checkmark$]
弹个通知提醒我 / “pop me a notification”	notify	rt1[REC, 11, $\times$] $\rightarrow$ rt2[REC, 11, $\times$] $\rightarrow$ rt3[EXT, 34, $\checkmark$]
查下这家公司背景 / “look up this company’s background”	enterprise_query	rt1[REC, 5, $\times$] $\rightarrow$ rt2[REC, 5, $\times$] $\rightarrow$ rt3[EXT, 31, $\checkmark$]
处理个文档 / “handle a document” (control)	file (core)	rt1[REC, 8, $\checkmark$] — reachable first attempt in both arms
搜点资料 / “search some material” (control)	search (core)	rt1[REC, 5, $\checkmark$] — reachable first attempt in both arms

different-provider detectors (qwen-max, glm-5); the merge-off exposure set is fixed by the rule intent and is therefore identical across detectors (46.0% on this subsample). All three detectors recognize the correct tool at high rates (82–98%), and the gap persists for every one of them (42–52%): because exposure is held fixed, the gap simply tracks each detector’s recognition rate, confirming that the loss is produced by the exposure stage, not by any single model. Notably, the stricter glm-5 (mean 1.0 recommended tool vs. 1.9 for the primary) still leaves a 42.0% gap.

Reference reproduction on open-source strategies (Table 6). Implementing three common exposure strategies under the same round2 recognition signal, an intent-router with a static allow-list reproduces WeClaw’s merge-off coverage (42.2%) and gap (56.2%) *exactly*, and its gap collapses to 0.0% once the recognized tools are merged; an all-tools full-bind has no gap but exposes all 125 tools per query (the cost the tiering was

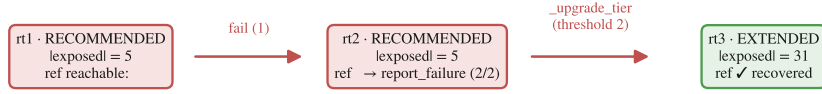
meant to avoid); and a Top-K tool retriever sits in between (coverage 69.2%, gap 30.6%, closing to 0.0% under merge). Thus the gap is a property of the two-stage decoupling (recognition not merged into construction), not of WeClaw-specific code.

6 | RELATED WORK

Closest neighbors (we differentiate directly). Five works are rhetorically or architecturally adjacent and must be addressed head-on. (i) The **Knowing-Doing Gap** [8] is the most similar in shape—“mismatches persist even at high meta-cognitive confidence”—and also splits tool use into cognition/execution stages, calling for “architectural modifications to bridge cognition-action representations.” However, it differs from ours on six axes: the fault location (model hidden-state representation geometry vs. an interface between two *harness*

Query: "what time is it now" (ref = datetime_tool)

BEFORE (merge off):



2 failed round-trips + 1 escalation ≈ 8.3 s added latency (measured)

AFTER (merge on):



0 failed round-trips · 0 escalations · 0 added latency · ref merged in place (setl 5→6 vs detour to 31)

FIGURE 4 Representative before-arm trigger chain for “what time is it now” (ref = datetime_tool): two failed round-trips inside the RECOMMENDED tier (lexposedl = 5, ref absent), then one escalation to EXTENDED (lexposedl = 31) where the tool finally becomes reachable; after the fix the tool is reachable on the first attempt.

TABLE 4 Real end-to-end (live) run, $n = 500$, every round a genuine deepseek-v4-flash function-calling call; latency is measured accumulated per-round-trip model time (concurrency-independent). The live arm reproduces the offline/replay first-attempt split under real API nondeterminism.

Metric	before (merge off)	after (merge on)
First-attempt reachability	42.2% [37.9, 46.6]	98.4% [96.9, 99.2]
Final reachability	100%	100%
Model first call = ref	71.0% [66.9, 74.8]	72.8% [68.7, 76.5]
Mean round-trips / query	2.58	1.03
Mean accumulated latency	11,213 ms	4,251 ms
Median accumulated latency	7,428 ms	1,375 ms
Total LLM calls	1,288	517

TABLE 5 Multi-detector replication on a 150-query stratified subsample. The merge-off exposure is detector-independent (rule intent, 46.0%); the gap persists across three heterogeneous detectors.

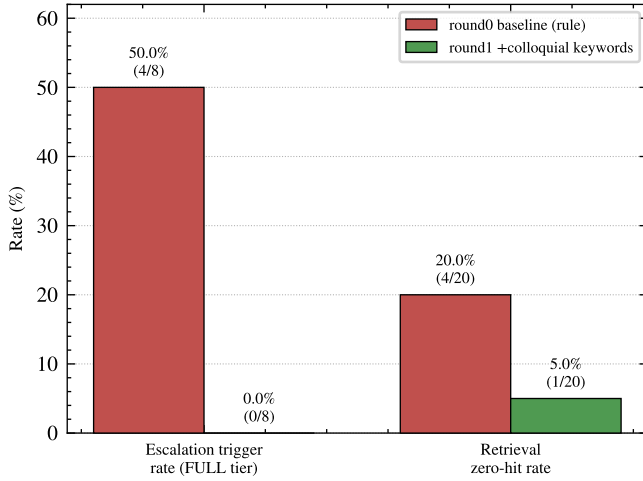
Detector (provider)	rec_hit	exposed	gap [95% CI]
deepseek-v4-flash (primary)	98.0%	46.0%	52.0% [44.1, 59.8]
qwen-max (Alibaba)	95.3%	46.0%	51.3% [43.4, 59.2]
glm-5 (Zhipu)	82.0%	46.0%	42.0% [34.4, 50.0]

software modules); the failure timing (the model *sees* the tool yet does not call it vs. the tool *never enters* the visible set, i.e., our failure is strictly upstream); the diagnosis (white-box

TABLE 6 Reference reproduction across three common open-source exposure strategies ($n = 500$, deterministic, same recognition signal as round2). Only the decoupled intent-router reproduces WeClaw’s gap; merging recognized tools closes it in every strategy.

Strategy	merge	cov.	gap	expo
intent-router (allow-list)	off	42.2%	56.2%	12.1
intent-router (allow-list)	on	98.4%	0.0%	13.1
all-tools full-bind	—	100%	0.0%	125
Top-K retriever	off	69.2%	30.6%	15.2
Top-K retriever	on	99.8%	0.0%	15.8

linear probing vs. black-box pipeline coverage audit); the fix (auxiliary loss / model change, requiring training vs. zero-training remapping); the trigger (an intrinsic property across four models vs. *detector-upgrade-triggered*); and the metric (mismatch rate vs. exposure-coverage gap). (ii) **“How Many Tools Should an LLM Agent See?”** [9] states “show too few and the correct tool may not appear” and learns per-query shortlist depth by RL; it attributes under-exposure to *shortlist depth (K too small)*, whereas we show that **even with sufficient K and 100% correct recognition**, the construction stage silently drops tools—a failure depth-tuning cannot fix in principle. (iii) **The 99% Success Paradox** [10] shows “selectivity vanishes even with perfect selectors,” but is an *evaluation-methodology critique* (report a chance-corrected metric); ours is a *systems causal diagnosis* plus a verifiable fix (gap $\rightarrow$ 0%).



† round1 keywords mirror eval query wording. 0% may overfit; hold-out validation required before dropping floor.

FIGURE 5 (supplementary) FULL-tier floor ablation (round0 vs. round1, rule-based intent). Removing the extended-tool floor makes 50% of ambiguous FULL-tier queries escalate (4/8) and leaves a 20% lexical-retrieval zero-hit rate (4/20); back-filling colloquial keywords into tool descriptions drops both to 0% (0/8) and 5% (1/20). The floor-removal result is wording-independent and isolates the mechanism (which tools enter the constructed set); the keyword back-fill is an upper bound whose overlap with the evaluation wording is noted in the Limitations.

(iv) **ITR / Dynamic System Instructions and Tool Exposure** [1] is the architectural prototype of confidence-gated exposure and reports only positive gains (context −95%, routing +32%); it never audits the “recognized vs. actually exposed” coverage gap. We position our work as a *failure-mode audit of ITR-class confidence-gated exposure*. (v) **AgentWeave: Routing Before Reasoning** [2] is closest architecturally—“candidate-space construction can materially affect a fixed model’s function-calling behavior”—and reports a *positive* result (deterministic routing beats all-tools/random/semantic baselines); ours is the *negative* counterpart (a detector upgrade silently breaks construction). In fact, its finding that semantic top-8 = 0/48 corroborates ours: semantic ranking alone does not construct a usable exposure set.

The named-“gap” family. Several gaps have been named for tool use, all on the *model or content side*: the adoption gap (tools available but unused) [11], the knowing-doing gap [8], the prerequisite gap (retrieved but missing dependencies) [12], query-skill misalignment [13], and knowledge-retrieval disassociation [14]. In fact, none is caused by *two decoupled modules*

inside the harness. Ours is the only gap localized at the **pipeline interface** and the only one shown to occur **even when recognition is near-perfect (98.2%)**.

Disciplinary anchor. A recent survey [6] on agent system and harness design asks the question of “where does the bottleneck reside—in the foundation model, in the execution harness, or in the coupling between them?” and lists model-harness co-evolution as an open challenge. Thus, we position the Recognition-Exposure Gap as, to our knowledge, the **first quantified tool-exposure instance of that coupling bottleneck**: the model is not wrong, the harness modules are each individually correct, and the failure lives entirely in their coupling.

Methodological allies and the counterintuitive framing. PluginEval [15] separates generation from verification and attributes errors stage-by-stage; our separation of recognition from exposure follows the same logic and supports our experimental design’s validity. RubricRefine [16] and PAEF [17] both stress failures that “run to completion without raising errors” / evade standard metrics—echoing our “silent reduction.” The mainstream consensus that *fewer tools is better* [3,4] and that *ranking does not decide how many to select* [18] frames our contribution: narrowing is right, but narrowing **silently past the point of correctness** is a distinct hazard. On the low tool-invocation rates reported for computer-use agents (36.3% in OSWorld-MCP) [19], usually attributed to model capability/willingness, our result offers an alternative partial explanation: some fraction may stem from exposure under-coverage.

Foundational anchors. Exposure budgeting has classical roots. Candidate ranking under a fixed display budget—deciding which few items to surface—is a long-standing information-retrieval problem [20]; the “tool learning” framing of an LLM selectively invoking external APIs [21] and large-scale API grounding [22] are the modern descendants of the route-before-reasoning family [3,4] our harness instantiates. These lines tune *which and how many* tools to surface by ranking or selection; none separates *recognition* from *exposure* and measures the divergence between them, which is the gap this work names.

7 | DISCUSSION AND LIMITATIONS

Why it matters. The gap inverts a natural engineering instinct. Teams upgrade intent detectors expecting strictly better tool access. However, under confidence-tiered exposure with decoupled injection, the upgrade can *reduce* availability while every recognition metric improves. Because the failure is silent (tasks still recover via the escalation ladder), it can persist in production while dashboards stay green—much like the

“silent” collapses reported for agent memory [23] and evaluation [17]. The fix is cheap and structural: connect the two stages.

A reusable audit. The three probes instantiate a transferable checklist any confidence-tiered harness can run given only a per-query ref: (i) measure recognition ($P(\text{rec_hit})$) and exposure ($P(\text{exposed})$) on the *same* queries and report their divergence, not recognition alone; (ii) treat the gap event $\text{rec_hit} \wedge \neg \text{exposed}$ as a first-class metric with a confidence interval; (iii) drive the escalation ladder (deterministically, then live) to price the gap in failed round-trips and latency; and (iv) re-run (i)–(ii) after *any* detector or confidence-mapping change, since a recognition improvement is exactly what can widen the gap. Steps (i)–(ii) alone already surface the failure in all three open-source baselines (Section 5.8), suggesting the audit is portable beyond WeClaw.

Limitations. (1) *Scope of the sample*: we evaluate 500 by-construction queries over 25 tools within a single production desktop agent and a single language (colloquial Chinese); the queries are seeded rather than drawn from live traffic, so the absolute gap rate may shift under a different tool mix or query distribution, though the mechanism itself is distribution-independent. (2) *Framework reproduction*: we abstract a framework-independent condition and reproduce the same two-stage decoupling inside three common open-source exposure strategies (a reference implementation, not a deployment of those frameworks); a live reproduction on an external agent framework remains future work. (3) *Deterministic replay and live run*: round3 injects round2’s measured recommendation (zero cost, fully reproducible) and its after-arm exposure (merging only ref) is a lower bound; the live run removes the fixed-recognition idealization and measures real wall-clock, but reports accumulated model latency (excluding tool execution and second-pass generation) to remain concurrency-independent. (4) *Detectors*: recognition is replicated across three heterogeneous LLM detectors (deepseek-v4-flash, qwen-max, glm-5) and the merge-off exposure set is detector-independent by construction, but all three are closed commercial models; rule/hybrid and open-weight detectors would broaden the evidence. (5) *Ablation hold-out*: the round1 keyword back-fill reuses evaluation wording, so its 0% escalation is an upper bound pending a held-out query split; the wording-independent floor-removal result (Section 5.4) is unaffected. None of these limitations affects the core structural claim, which follows from the deterministic code path (confidence 0.9 $\rightarrow$ narrowest tier; recommended tools not merged) and is cross-checked by three complementary probes (offline counterfactual, deterministic replay, and live end-to-end).

8 | CONCLUSION

We identified, named, and quantified the **Recognition-Exposure Gap**: in confidence-tiered LLM agents, upgrading the intent detector can silently reduce tool availability because confidence-to-tier mapping and recommended-tool-to-exposure-set construction are decoupled. On a production desktop agent, over 500 by-construction queries a detector with 98.2% recommendation accuracy exposed only 42.2% of the recognized tools (gap 56.2%); a deterministic replay and a real end-to-end run showed the gap costs 788 failed round-trips, 499 escalations, a $3.7\times$ exposure inflation, and $2.6\times$ per-query latency, all while recovering “correctly but late.” A zero-training co-design fix—an existence proof that the gap is closable at construction time—closes it (first-attempt reachability 42.2% $\rightarrow$ 98.4% live) and reduces exposure. In fact, the gap is, to our knowledge, the first quantified tool-exposure instance of the model-harness coupling bottleneck, and its failure condition—confidence-tiered exposure plus decoupled recommended-tool injection—is defined detector- and framework-independently, supported by blind second annotation ($\kappa = 0.987$), three-detector replication, and a reference reproduction across three open-source exposure strategies. We hope the naming and the audit method prompt broader coverage-vs-recognition reporting for tool-exposure systems.

REPRODUCIBILITY STATEMENT

All quantitative results are produced by the unmodified production code paths of the studied desktop agent—`_determine_tier`, `_get_tool_names_for_tier`, `report_failure`, `_upgrade_tier`, and the `_augment_with_llm_recommended` fix—on a fixed source snapshot (git tag `n1-eval-v1`). The 500 evaluation queries, their by-construction ground-truth ref tools, the escalation threshold (2), and the deterministic-replay latency proxy (1,950 ms) are fully specified in Section 5. The offline counterfactual (round2) is a real LLM measurement over all 500 queries with Wilson 95% CIs; the deterministic escalation-ladder replay (round3) is *deterministic*, injecting round2’s measured recommendation so exposure construction and escalation reproduce exactly at zero API cost; a separate *live* run re-calls the LLM every round and measures wall-clock. Recognition labels are cross-checked by blind second annotation (Cohen’s $\kappa = 0.987$), replicated across three heterogeneous detectors, and reproduced inside three open-source exposure strategies. Per-run artifacts (queries, labels, and all outputs) are released in the anonymized repository.

DATA AND CODE AVAILABILITY

The studied agent is open source. For the duration of the double-blind review, the tool-exposure engine, both intent detectors, the replay harness, and the round0–round3 evaluation artifacts (seed queries, labels, and per-run outputs) are available in an anonymized repository at <https://anonymous.4open.science/r/weclaw-n1-etri-1737>. Data that could reveal local user identity, credentials, or private runtime paths has been excluded or normalized before release.

REFERENCES

- [1] U. Franko, *Dynamic system instructions and tool exposure for efficient agentic LLMs*, arXiv preprint arXiv:2602.17046 (2025), .
- [2] S. Singla et al., *AgentWeave: Routing before reasoning for efficient function calling in tool-rich language models*, arXiv preprint arXiv:2608.23078 (2026).
- [3] T. Gan et al., *RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation*, arXiv preprint arXiv:2505.03275 (2025).
- [4] V. Paramanayakam et al., *Less is more: Optimizing function calling for LLM execution on edge devices*, arXiv preprint arXiv:2411.15399 (2024).
- [5] Anonymous, *WeClaw: An open-source desktop LLM agent*, <https://anonymous.4open.science/r/weclaw-n1-etri-1737> (2026).
- [6] J. Guo, Z. Hao, and C. Wang, *From question answering to task completion: A survey on agent system and harness design*, arXiv preprint arXiv:2606.20683 (2026).
- [7] Y. Lin et al., *AgentRaft: Automated detection of data over-exposure in LLM agents*, arXiv preprint arXiv:2603.07557 (2026).
- [8] Y. Cheng et al., *Model-adaptive tool necessity reveals the Knowing-Doing Gap in LLM tool use*, arXiv preprint arXiv:2605.14038 (2026).
- [9] V. Repantis et al., *How many tools should an LLM agent see? a chance-corrected answer*, arXiv preprint arXiv:2605.24660 (2026), .
- [10] V. Repantis et al., *The 99% success paradox: When near-perfect retrieval equals random selection*, arXiv preprint arXiv:2605.18857 (2026).
- [11] S. Fan et al., *Screenshots or tools? eliciting tool use and managing multimodal context in hybrid GUI-MCP computer-use agents*, arXiv preprint arXiv:2608.03327 (2026).
- [12] D. Liu et al., *Graph-of-Skills: Dependency-aware structural retrieval for massive agent skills*, arXiv preprint arXiv:2604.05333 (2026). (Accepted to EMNLP 2026, Main Conference.)
- [13] S. Liu, Y. Yang, H. Xiao, M. Yang, and X. Peng, *Imagine Before Retrieval: Prospective skill retrieval for LLM agents*, arXiv preprint arXiv:2609.01642 (2026).
- [14] A. Hathidara et al., *ToolSense: A diagnostic framework for auditing parametric tool knowledge in LLMs*, arXiv preprint arXiv:2606.12451 (2026).
- [15] D. Xu et al., *PluginEval: A diagnostic benchmark for fine-grained error attribution in function calling*, arXiv preprint arXiv:2608.08700 (2026).
- [16] W. LeVine, B. Evers, S. Saltwick, and A. Venkatesh, *RubricRefine: Improving tool-use agent reliability with training-free pre-execution refinement*, arXiv preprint arXiv:2605.09730 (2026).
- [17] M. Pandey, *Evaluating agentic AI in the wild: Failure modes, drift patterns, and a production evaluation framework*, arXiv preprint arXiv:2605.01604 (2026).
- [18] Y. Feng et al., *Scores are not decisions: Cost-aware stopping for tool acquisition in LLM agents*, arXiv preprint arXiv:2607.27083 (2026), .
- [19] H. Jia et al., *OSWorld-MCP: Benchmarking MCP tool invocation in computer-use agents*, arXiv preprint arXiv:2510.24563 (2025).
- [20] G. Salton and C. Buckley, *Term-weighting approaches in automatic text retrieval*, Information Processing & Management **24**(5), 513–523 (1988).
- [21] Y. Qin et al., *Tool learning with foundation models*, arXiv preprint arXiv:2304.08354 (2023).
- [22] K. Y. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, *Gorilla: Large language model connected with massive APIs*, arXiv preprint arXiv:2305.15334 (2023).
- [23] Anonymous, *EBEAC: Event-driven Zero-LLM-Cost experience capture for agent memory*, manuscript submitted for publication (author and manuscript identity withheld for double-blind review) (2026).